JavaScript Foundations Lab — Currículo

Objetivo

Consolidar os fundamentos de JavaScript para resolver problemas, construir interfaces web e avançar com segurança para Node.js, APIs e desenvolvimento backend.

Como estudar

1. Estudar os tópicos de cada módulo em sequência e experimentar conceitos pontualmente quando necessário.
2. Resolver um exercício integrador por módulo, aplicando os tópicos relacionados.
3. Verificar as competências, executar testes ou cenários de validação e corrigir falhas antes de avançar.

Critério geral: saber explicar e aplicar os conceitos com autonomia, sem exigir domínio de detalhes avançados que não influenciam o trabalho atual.

Ambientes: até JS-07, os exercícios podem ser executados no console do navegador ou no Node.js, conforme indicado; JS-08 e JS-09 requerem navegador para DOM e Web APIs. JavaScript é a linguagem; document, fetch e APIs de arquivos pertencem aos respectivos ambientes.

Foco profissional: compreender dados, funções, controle de fluxo, módulos, tratamento de erros, testes, assincronismo e HTTP antes de avançar para Node.js. DOM e APIs do navegador são essenciais para integrar interfaces, mas não substituem os fundamentos da linguagem. Recursos avançados sem uso imediato ficam como consulta, não como pré-requisito.

Mapa curricular

JS-01 — Valores, Operadores e Controle de Fluxo - Simulador de tarifas
JS-02 — Funções, Escopo e Closures - Calculadora de regras de negócio
JS-03 — Coleções e Processamento de Dados - Relatório de pedidos
JS-04 — Objetos, Métodos e Classes - Gestor de catálogo
JS-05 — Sintaxe Prática e Módulos - Catálogo modular
JS-06 — Erros, Depuração e Testes - Validador de registros
JS-07 — Assincronismo e Modelo de Execução - Simulador de tarefas assíncronas
JS-08 — DOM e Eventos - Painel interativo de tarefas
JS-09 — HTTP, JSON e Integração Web - Consulta de dados de uma API

JS-01 — Valores, Operadores e Controle de Fluxo

Objetivo: Escrever programas simples, prever o resultado de expressões e controlar decisões e repetições.

Pré-requisito: HTML básico e noções de programação.

JS-01.01 — Sintaxe, valores e declarações

- Execução de JavaScript em navegador e Node.js; console, arquivos .js, comentários, blocos e distinção entre linguagem e APIs do ambiente.
- Tipos primitivos (string, number, boolean, undefined, null, bigint, symbol) e objetos; literais e typeof, incluindo a particularidade de typeof null.
- Números em JavaScript: NaN, Infinity, conversão com Number(), verificação com Number.isNaN()/Number.isFinite() e cálculos úteis com Math; reconhecer limites do tipo number para valores monetários.
- Bindings const, let e conhecimento de var; reatribuição, valores primitivos e const não significar imutabilidade de objetos.

JS-01.02 — Expressões, operadores e decisões

- Operadores aritméticos, de atribuição, comparação (===, !==, <, >) e lógicos (&&, ||, !); precedência e curto-circuito.
- Conversão explícita, coerção, truthy/falsy, diferença entre == e ===; template literals.
- Decisões com if, else, switch e operador condicional ?:; condições legíveis e validação de entradas simples.

JS-01.03 — Laços e controle de execução

- for, while e do...while; atualização de contadores, condições de parada e prevenção de loops infinitos.
- break, continue, blocos e combinação de decisões com repetição.
- Escolha de um laço apropriado para repetir cálculos ou inspecionar uma sequência de valores.

Competências

- Explicar valores, tipos, coerção e o resultado de expressões comuns.
- Criar decisões e laços corretamente, sem depender de copiar soluções prontas.
- Organizar uma rotina pequena e diagnosticar erros de sintaxe e de lógica.

Exercício integrador

Simulador de tarifas. Calcular preços finais para diferentes quantidades e condições de desconto, gerando um relatório no console por meio de decisões e laços.

Critérios de conclusão:

- Executar com cenários variados e explicar os cálculos.
- Usar condições de parada corretas e evitar comparações com coerção desnecessária.
- Não exigir HTML, DOM ou bibliotecas externas.

Ambiente: Console do navegador ou Node.js; sem interface obrigatória.

Consultar módulo

JS-02 — Funções, Escopo e Closures

Objetivo: Decompor problemas em funções reutilizáveis e compreender a resolução de nomes e o estado de funções.

Pré-requisito: JS-01 e seus fundamentos essenciais.

JS-02.01 — Funções e composição

- Declaração, expressão e arrow function; parâmetros, argumentos, retornos, parâmetros padrão e diferenças entre return e efeitos colaterais.
- Divisão em funções pequenas com responsabilidades claras; validação de parâmetros e composição de resultados.
- Funções como valores: armazenar, passar como argumento e receber como retorno.

JS-02.02 — Escopo léxico e closures

- Escopo de bloco, de função e de módulo; shadowing e acesso a variáveis externas.
- Hoisting e temporal dead zone para entender usos antes da declaração; diferenças práticas entre var, let e const.
- Closure como função que mantém acesso ao ambiente léxico; encapsulamento simples sem exigir padrões avançados.

Competências

- Implementar funções independentes que recebem dados e devolvem resultados.
- Prever visibilidade de variáveis em blocos, funções e closures.
- Separar regras de negócio de entrada e saída de dados.

Exercício integrador

Calculadora de regras de negócio. Implementar cálculos e validações por funções; criar uma função configuradora que retorna outra função para praticar closure.

Critérios de conclusão:

- As regras possuem parâmetros e retornos claros.
- Existe ao menos um caso justificável de closure, sem estado global desnecessário.
- As funções são executadas com casos válidos e inválidos.

Ambiente: Console do navegador ou Node.js.

Consultar módulo

JS-03 — Coleções e Processamento de Dados

Objetivo: Representar, percorrer e transformar conjuntos de informações com clareza e atenção à mutabilidade.

Pré-requisito: JS-02 e seus fundamentos essenciais.

JS-03.01 — Arrays, strings e iteração

- Arrays: criação, índices, acesso, tamanho, inserção e remoção; métodos essenciais de string (trim, includes, slice, split).
- Iteração com for...of, forEach e laços convencionais; diferenças entre valores e índices.

- Operações que mutam a coleção e operações que produzem novos valores; evitar alterações inesperadas.

JS-03.02 — Transformação e agregação

- map, filter, find, some, every e reduce: escolher pelo resultado desejado, não por preferência estética.
- Ordenação com sort e função comparadora, cópia com slice ou spread; cuidado com referências compartilhadas.
- Noções de Set para valores únicos e Map para associações chave–valor; iteráveis personalizados e geradores ficam para consulta posterior.
- Fundamentos de datas em aplicações: Date, timestamp, formato ISO 8601 e diferenças entre horário local e UTC; usar Intl.DateTimeFormat quando a apresentação exigir localização, sem aprofundar fusos horários neste Lab.

Competências

- Escolher uma representação e um método de coleção adequados ao problema.
- Produzir relatórios por transformação, filtragem e agregação sem loops desnecessariamente complexos.
- Explicar quando uma operação modifica os dados originais.

Exercício integrador

Relatório de pedidos. Processar uma coleção de pedidos para filtrar estados, calcular totais, localizar registros e produzir um resumo.

Critérios de conclusão:

- Separar obtenção, transformação e apresentação dos dados.
- Utilizar corretamente operações imutáveis quando a coleção original deve permanecer intacta.
- Testar coleções vazias e dados não encontrados.

Ambiente: Console do navegador ou Node.js.

Consultar módulo

JS-04 — Objetos, Métodos e Classes

Objetivo: Modelar entidades e compreender identidade, referências, métodos e a base da orientação a objetos em JavaScript.

Pré-requisito: JS-03 e seus fundamentos essenciais.

JS-04.01 — Objetos, propriedades e referências

- Criação com literais, leitura e escrita com ponto/colchetes, propriedades calculadas, operações com Object.keys, Object.values e Object.entries.
- Identidade de objetos, comparação por referência, mutação, cópias superficiais e diferença entre cópia e compartilhamento de referências.
- Destructuring de objetos, ... para cópias superficiais e propriedades opcionais com ?.; usar ?? para valores ausentes sem substituir 0 ou string vazia. Symbol como chave especial é conteúdo de consulta, não requisito inicial.

JS-04.02 — Métodos e contexto de this

- Métodos como funções associadas a propriedades e valor de this determinado pela forma de chamada em funções comuns.
- Diferença para o this léxico de arrow functions; risco de perder contexto ao passar métodos como callbacks.
- Quando preferir funções puras a objetos com estado; organização de comportamento sem acoplamento desnecessário.

JS-04.03 — Protótipos e classes essenciais

- Cadeia de protótipos para procura de propriedades e métodos; relação entre classes JavaScript e protótipos.
- class, constructor, métodos de instância e new; introdução criteriosa a extends e super.
- Usar classes quando agregarem clareza; não exigir reescrever classes em cadeias manuais de construtores ou dominar herança complexa.

Competências

- Modelar dados e distinguir referência de cópia em objetos e arrays.
- Criar métodos e explicar o comportamento de this em chamadas comuns.
- Reconhecer o funcionamento básico de classes e protótipos e escolher abstrações proporcionais.

Exercício integrador

Gestor de catálogo. Implementar um catálogo simples com entidades, operações de atualização e métodos para consultar ou modificar registros.

Critérios de conclusão:

- Evitar mutações acidentais e acessar propriedades de maneira segura.
- Justificar o uso de objetos literais ou de uma classe simples.
- Não criar hierarquias de herança artificiais.

Ambiente: Console do navegador ou Node.js.

Consultar módulo

JS-05 — Sintaxe Prática e Módulos

Objetivo: Utilizar recursos modernos da linguagem e dividir código em módulos com responsabilidades definidas.

Pré-requisito: JS-04 e seus fundamentos essenciais.

JS-05.01 — Aplicação de recursos modernos

- Consolidar, em uma implementação curta, recursos já introduzidos: template literals (JS-01), parâmetros padrão (JS-02), spread, destructuring, ?. e ?? (JS-03/04). O objetivo é aplicar, não repetir aulas isoladas.
- Usar rest parameters (...args) e propriedades abreviadas quando tornarem funções e objetos mais claros; distinguir || de ?? nos valores padrão.

- Evitar sintaxe ornamental, cópias superficiais confundidas com profundas e operações com efeitos colaterais difíceis de rastrear.

JS-05.02 — Módulos ECMAScript

- export nomeado e default, import, dependências entre arquivos e escopo de módulo.
- Configuração básica de módulos no navegador (type="module") e no Node.js (ESM); distinguir resolução de módulos do código da linguagem. Conhecer package.json, scripts npm e a diferença entre instalar dependências e escrever o próprio código, sem exigir bibliotecas externas.
- Divisão entre entrada, regras, utilitários e dados; cuidado com dependências circulares.

Competências

- Reutilizar a sintaxe moderna aprendida anteriormente para melhorar legibilidade, sem estudá-la novamente como conteúdo separado.
- Dividir uma solução em módulos independentes e importá-los corretamente.
- Executar uma pequena aplicação modular e explicar suas dependências.

Exercício integrador

Catálogo modular. Distribuir funções e dados de um catálogo por módulos, usando importações explícitas e recursos modernos de acesso e composição.

Critérios de conclusão:

- Cada módulo possui responsabilidade identificável.
- Evitar dependências circulares e duplicação de regras.
- Executar o conjunto no ambiente escolhido e registrar o procedimento.

Ambiente: Preferencialmente Node.js ou navegador com suporte a ESM.

Consultar módulo

JS-06 — Erros, Depuração e Testes

Objetivo: Tratar falhas de forma previsível e verificar o comportamento do código com casos reproduzíveis.

Pré-requisito: JS-05 e seus fundamentos essenciais.

JS-06.01 — Erros, exceções e validação

- Erros de sintaxe, de execução e de lógica; objeto Error, throw, try...catch...finally e propagação de exceções.
- Validação de parâmetros e mensagens de erro úteis, sem capturar exceções indiscriminadamente.
- Diferença entre erro de programação e resultado de domínio esperado, como busca sem correspondência.

JS-06.02 — Depuração e testes fundamentais

- Leitura de stack trace, console, breakpoints no DevTools ou depurador do runtime e inspeção das entradas que provocam erro.

- Casos positivos, negativos e limites; testes automatizados básicos com node:test/node:assert quando Node.js estiver disponível, sem dependência de frameworks.
- Separação de lógica e efeitos para facilitar testes; evitar testes que apenas repetem a implementação.

Competências

- Lançar e tratar exceções sem ocultar problemas reais.
- Investigar erros usando stack trace e ferramentas de depuração.
- Escrever verificações reproduzíveis para comportamentos esperados.

Exercício integrador

Validador de registros. Validar entradas de um conjunto de registros e apresentar erros inteligíveis; incluir testes de casos válidos, inválidos e extremos.

Critérios de conclusão:

- As validações possuem mensagens compreensíveis.
- Erros não são silenciados; casos de domínio são tratados conscientemente.
- Testes executam sem serviços externos.

Ambiente: Node.js com módulo de testes nativo ou verificações automatizadas simples.

Consultar módulo

JS-07 — Assincronismo e Modelo de Execução

Objetivo: Coordenar tarefas assíncronas, tratar falhas e compreender a ordem de execução em alto nível.

Pré-requisito: JS-06 e seus fundamentos essenciais.

JS-07.01 — Callbacks e Promises

- Código síncrono e callbacks; APIs de temporização como exemplo de operação fornecida pelo ambiente.
- Ciclo de vida de Promises, then, catch e finally; encadeamento e propagação de falhas.
- Composição com Promise.all e Promise.allSettled quando houver tarefas independentes.

JS-07.02 — Funções async e await

- Funções async, uso de await, tratamento de erros com try...catch e retorno de Promises.
- Diferença entre execução sequencial e concorrente; evitar await serial desnecessário.
- Cancelar ou controlar operações é responsabilidade da API e do ambiente; reconhecimento básico de AbortController.

JS-07.03 — Event loop e ordem de execução

- Pilha de chamadas, tarefas e microtarefas; relacionamento entre execução síncrona, timers e reações de Promises.
- Exercícios simples de ordenação; compreender que o modelo de agendamento depende do ambiente de execução.
- Não confundir assíncrono com execução paralela automática de JavaScript.

Competências

- Implementar fluxos assíncronos com Promises e async/await.
- Tratar rejeições e escolher entre sequência e concorrência.
- Explicar resultados de ordenação de tarefas sem depender de detalhes internos avançados.

Exercício integrador

Simulador de tarefas assíncronas. Orquestrar tarefas simuladas com tempos e resultados diferentes, informar andamento e tratar falhas com Promises.

Critérios de conclusão:

- Executar tarefas sequenciais e concorrentes quando apropriado.
- Tratar rejeições sem deixar Promises pendentes sem observação.
- Prever a ordem dos logs síncronos, de microtarefas e de timers simples.

Ambiente: Node.js ou navegador; sem dependência de API externa.

Consultar módulo

JS-08 — DOM e Eventos

Objetivo: Conectar a lógica JavaScript à interface HTML respeitando estrutura, eventos e acessibilidade.

Pré-requisito: JS-07 e seus fundamentos essenciais.

JS-08.01 — Document, DOM e manipulação

- Diferença entre JavaScript, objetos do navegador (window, document) e DOM.
- Consulta com querySelector e querySelectorAll, propriedades e atributos; textContent, createElement, append e remoção.
- Manipulação de classes e estados sem substituir a semântica HTML; preferir textContent para texto não confiável.

JS-08.02 — Eventos, formulários e estado de interface

- addEventListener, objeto de evento, propagação, delegação e preventDefault() quando pertinente.
- Leitura de campos, eventos input, change e submit, FormData e preventDefault() quando pertinente; validação e mensagens compreensíveis para o usuário.
- Atualização consistente de listas, contadores e estados vazios; cuidados com foco e feedback acessível. Priorizar textContent e criação de nós ao exibir dados externos, evitando inserir dados não confiáveis por innerHTML.

Competências

- Selecionar, criar e atualizar elementos do DOM corretamente.
- Responder a eventos e processar dados de formulários.
- Separar regras de negócio da manipulação do documento.

Exercício integrador

Painel interativo de tarefas. Criar página com formulário e lista filtrável, permitindo adicionar, atualizar e remover tarefas sem recarregar o documento.

Critérios de conclusão:

- Os controles podem ser usados com mouse e teclado.
- Estado, contagem e lista permanecem sincronizados.
- O código separa dados, eventos e renderização.

Ambiente: Navegador com HTML e CSS; JavaScript responsável pelo comportamento.

Consultar módulo

JS-09 — HTTP, JSON e Integração Web

Objetivo: Consumir serviços web e transformar respostas em dados e interfaces, formando a ponte para APIs e backend.

Pré-requisito: JS-08 e seus fundamentos essenciais.

JS-09.01 — HTTP, fetch e JSON

- Diferença entre JavaScript e a Web API fetch; visão prática de URL, requisição/resposta, métodos GET/POST, status HTTP, cabeçalhos e corpo. Construir parâmetros com URL/URLSearchParams em vez de concatenar strings sem cuidado.
- fetch, objeto Response, verificação de response.ok, corpo JSON e distinção entre erros de rede e status HTTP de erro.
- JSON.parse/JSON.stringify, conversão entre texto JSON e valores JavaScript; noções de CORS e limites de segurança no navegador. Nunca incluir segredos privados (tokens de servidor, senhas de banco) no código entregue ao navegador.

JS-09.02 — Integração e estados de interface

- Organização do fluxo de requisição, transformação de dados e apresentação no DOM.
- Tratamento de carregamento, sucesso, lista vazia, erros e atualização do estado de interface.
- Separar camada de acesso a dados da apresentação, evitando acoplamento a detalhes de uma API. Reconhecer localStorage como armazenamento local simples, opcional, e não como banco de dados ou lugar seguro para segredos.

Competências

- Consumir dados JSON com fetch, utilizando URLs e parâmetros corretamente e tratando status HTTP, erros de rede e JSON inválido.
- Transformar respostas em informações úteis e renderizá-las em interface.
- Explicar o caminho frontend - HTTP - serviço - resposta e a necessidade de um backend real.

Exercício integrador

Consulta de dados de uma API. Implementar página que consulta um endpoint de teste e apresenta resultados com estados de carregamento, sucesso e erro.

Critérios de conclusão:

- Verificar response.ok e tratar JSON inválido ou resultados vazios.
- Conservar a interface utilizável em falhas de rede.
- Separar requisição, processamento e renderização em funções ou módulos.

Ambiente: Navegador; endpoint documentado e acessível ou serviço de teste controlado.

Consultar módulo

Critérios transversais de conclusão

- Correção: explicar o código e prever resultados sem depender de tentativa e erro aleatória.
- Robustez: tratar entradas inválidas, coleções vazias, exceções e respostas de API com erro.
- Manutenção: organizar funções e módulos por responsabilidade, evitar duplicação e registrar como executar os exercícios.
- Verificação: testar casos normais e de borda; inspecionar DOM, rede e console quando houver interface web.
- Segurança inicial: não expor segredos no frontend e não inserir dados externos como HTML executável.

Aprofundamentos sob demanda

Após dominar os fundamentos, consultar somente quando necessário: detalhes de protótipos e herança complexa; Symbol e protocolos de iteração; geradores; expressões regulares avançadas; metaprogramação com Proxy/Reflect; programação orientada a objetos avançada; otimizações do event loop específicas de cada ambiente.

Preparação para backend

Ao concluir o percurso, o próximo passo é aplicar essas competências em Node.js, trabalhando com linha de comando, npm, módulos, sistema de arquivos, HTTP, APIs, persistência e testes mais abrangentes. Git, terminal, fundamentos de HTTP e SQL são conhecimentos complementares à trilha; não precisam virar módulos artificiais de JavaScript. Este Lab não exige criar um servidor ou banco de dados antes de estudar essas tecnologias.

Referências

- MDN — Guia JavaScript
- MDN — Fundamentos de JavaScript
- MDN — Referência JavaScript